
Table of Contents

Introduction	1.1
Arduino Code	1.2
Setup	1.2.1
Demo Software	1.2.2
Motors	1.2.3
NeoPixel LEDs	1.2.4
Light Sensors	1.2.5
Servos	1.2.6
Chirp	1.2.7
Rangefinder	1.2.8
Power	1.2.9
UART/Comms	1.2.10
Advanced	1.2.11
Raspberry Pi Setup	1.3
Initial Configuration	1.3.1
Pi Software Install	1.3.2

Spirit Rover - Code Reference

Reference Manual for Spirit Rover Code and Functions

Spirit Arduino Quick Reference

This section describes the setup and use of various code and functionality of the Spirit Rover when using the Arduino based processor to run code.

Spirit Arduino Quick Reference

Setup:

The Arduino based processor on Spirit is identical to the processor and code uploading process used on our Ringo2 and Wink2 robots. Please follow the setup instructions for Wink2/Ringo2 and you will be all set to upload code onto Spirit.

Visit the Wink2 Getting Started page and follow the instructions as if you were using the Wink2 robot.

<http://www.plumgeek.com/wink-getting-started.html>

You will install the Arduino software environment, a the USB driver to communicate with the robot, and a few software libraries. This setup process only needs to be done one time for a given computer. If you're already using a Wink2 or Ringo2 robot, no further setup is needed for Spirit.

Please read this entire Setup section carefully before starting.

- Arduino calls its code files "sketches". These are basically text files with code which is compiled into machine readable instructions which are then uploaded into the processor on the Spirit Rover.
- You can begin uploading code at any time (you don't have to cycle the power or do anything special to begin the process.
- Each time a sketch is uploaded, the previous sketch already loaded on the processor is erased and replaced by the new sketch you are uploading. There is no limit to how many times you can upload new sketches to the robot.

Programming Help:

Spirit is intended as an advanced robot for more advanced users, though it can certainly be used by novice programmers as well. The supporting documents for Spirit won't attempt to teach you how to program. Code examples and tutorials will be provided as we move forward with the project. If you are new to programming or Arduino, here are a few tips to help you in the correct direction.

- If you are very new to code and feeling a bit overwhelmed, we suggest reading our [Learn to Code lessons for Wink2](#). The code used on Spirit is nearly identical. These lessons describe basic programming concepts in a super simple way, and you don't

need to have a Wink2 robot to understand and learn from them.

<http://www.plumgeek.com/learn-to-code.html>

- The Arduino based processor on Spirit is basically an Arduino UNO board (the most popular kind of Arduino board).
- You can use any code on Spirit you would use on any other Arduino board. There are millions of tutorials on the use of Arduino on the internet, and the Arduino website provides an excellent code reference as well.
- You will use the functions described in this guide to perform the significant functionality of Spirit, but you will also use other bits of code common to Arduino such as `delay()` functions, and the normal `if`, `for`, `while`, and other control statements.
- Read over and experiment with the example code we will be posting on the main [Spirit web page](http://www.plumgeek.com/spirit.html) here: <http://www.plumgeek.com/spirit.html>

Base Sketch:

When writing your own code, you should always begin from the `Rover_BaseSketch`. This is a template Arduino program that already contains all the back-end code and functions you will use to control the robot. Write your code inside the `loop()` and `setup()` functions as with any other Arduino board.

Restoring Shipping Code:

If you want to restore your Spirit Rover to the program it was shipped with, download the `Rover_DemoSketch`. Open this sketch inside Arduino and upload it to the robot. You can also study this sketch to see how the example behaviors work. It is somewhat complicated but you can still pick out individual sections to see how they work.

Charging the Rover:

Spirit is charged via the USB cable when connected to your computer or a USB wall power adapter cube. When you plug in the USB cable, the battery will automatically begin charging. It will charge whether the robot is turn on or off, but it will charge faster if it is turned off. A charge indicator LED is located near the battery connector. It is lit while charging is in progress, and turns off when charging is complete. Charging will stop automatically when complete. A full charge takes about four hours to complete.

The Spirit Rover will power off automatically if the battery level gets too low.

Rover Power:

To power on the robot, press the PWR button briefly. The PWR LED should light up when powered on. To turn off the robot, press and hold the PWR button for a few seconds until the PWR LED begins to strobe. Once the PWR LED begins to strobe you can release the PWR button. The strobing PWR LED indicates the robot has entered its power down cycle. During this time the robot signals the attached Raspberry Pi computer that it is about to turn off. The several seconds of strobing allows the Raspberry Pi to perform an orderly shutdown before power is removed. This is necessary to prevent problems with the file system on the SD card on the Raspberry Pi.

Spirit Arduino Quick Reference

Demo Software:

.
Your Spirit Rover is pre-loaded with a demo example. Its functionality is described in this section.

The demo software basically has four different modes as follows:

1. Random actions triggered by an object or hand placed within 100mm of the ultrasonic rangefinder (these are Spirit's "eyes").
2. Random autonomous roaming.
3. Light Seeking.
4. Transparent / Disabled Mode (used when Raspberry Pi takes over control).

You can view the demo behaviors and a description of how to change between them in our video here:

<https://youtu.be/JnGBNoFt7Ok>

- Use the BTN button to index between behavior modes 1 to 3.
- When using the Raspberry Pi, the demo behaviors are disabled and the Arduino processor enters a transparent mode as soon as the Raspberry Pi sends any motor control code. From that point forward, the robot is controlled by the Raspberry Pi.
- Normal function of the demo behaviors is restored after resetting the Arduino processor or cycling power to the robot.

Spirit Arduino Quick Reference

Motors Control:

motors();

```
motors(leftSpeed,RightSpeed); //negative numbers go backward. Values accepted -255 to 255.  
0 stops motor.
```

This function controls Spirit's motors. Values range between -255 and 255. Two values are passed - the value for the left motor speed and the right motor speed. Negative numbers make the motor turn backward and positive numbers make the motor turn forward. Zero values stop the corresponding motor.

WARNING NOTE: Spirit does not automatically detect edges by default. Be careful running motors if Spirit is on a surface like a table or desktop so it doesn't drive off and fall.

Spirit Arduino Quick Reference

NeoPixels:

Spirit has 27 NeoPixel LEDs that can individually be set to any color. Each pixel can be controlled individually by number, or by the use of functions that set groups of pixels to different colors. Each pixel actually contains three individual lights - one each of red, green, and blue. Each of these red, green, and blue parameters of each pixel can be set individually. By mixing different values almost any color can be created. By setting each red, green, and blue to the same amount, white color is created.

The pixels are addressed as follows:

0 Center pixel in middle of Spirit Main Board

1 Left Eye

2 Right Eye

3~14 Left Wing Pixels

15~26 Right Wing Pixels

Each pixel, or group of pixels is adjusted by setting each of the red, green, and blue elements of each pixel to a value between 0 and 255, with zero being completely off, and 255 being maximum bright.

Important Notes:

- These pixels are extremely bright. It generally is not necessary to set the pixels anywhere near maximum brightness. Normally values of 150 or so make the pixels appear very bright. There is no harm in using larger values, but know the pixels will consume much more power (shorter battery life) at the brighter values.
- At maximum brightness, full white (all values set to 255), each pixel will draw about 60 milliamps.
- Setting all pixels to max bright full white will draw about $60 \text{ ma} * 27 \text{ pixels} = 1,620$ milliamps (which is 1.6 amps). The maximum current draw of the entire robot is about 2000 milliamps - much more than this will cause the robot to power off due to over-current. This won't damage anything, but be aware of this if the robot is powering off while you're making the LEDs very bright. Remember the Raspberry Pi will draw about 300 to 500 milliamps idling, and each motor will draw about 60 milliamps when running. Servo movement causes spikes above this as well, so it is possible to cause an over-current shutdown of the robot when using lots of pixels at maximum brightness.

- At more "normal" levels, where pixels are various colors (so all elements aren't on at once) and at more normal brightness levels, each pixel will draw an average of 5 to 20 milliamps.

setPixelRGB();

```
setPixelRGB(pixelAddress, redValue, greenValue, blueValue); //controls a specific pixel to  
a specific color setPixelRGB(0, 0, 50, 0); //make center pixel on main board green, 50  
brightness value setPixelRGB(0, 70, 0, 70); //make center pixel on main board purple  
(red+blue), 70 brightness value setPixelRGB(3, 70, 0, 70); //make first pixel on left wing  
purple (red+blue), 70 brightness value
```

This is the primary function used to control individual pixels. The function accepts four values. The first is the address of the pixel you are controlling, and the second, third, and fourth values are the red, green, and blue values for the pixel. It can be used in loops as well to iterate through different pixels to set many to different values. Turn off any given pixel by setting the red, green, and blue values to zero.

setAllPixelsRGB();

```
setAllPixelsRGB(redValue, greenValue, blueValue); //sets all pixels to a specific color  
setAllPixelsRGB(0, 50, 0); //make all pixels green, 50 brightness value setAllPixelsRGB(0,  
0, 0); //turn off all pixels
```

This is a function to easily set all pixels on the robot to a specific color. Setting all pixels to zero values is an easy way to turn off all pixels. Be aware that setting all pixels to a high brightness setting will draw higher current from the battery. Keep the important notes above in mind.

eyesOn();

```
eyesOn(redValue, greenValue, blueValue); //sets both eyes to a specific color eyesOn(0,  
50, 0); //make both eyes green, 50 brightness value eyesOn(70, 0, 70); //make both eyes  
purple (red+blue), 70 brightness value eyesOn(0, 0, 0); //turn off both eyes
```

This is a quick function to set both eyes to the same color.

eyesOff();

```
eyesOff(); //turn off both eyes
```

This is a quick function to turn off both eyes.

rightEye();

```
rightEye(redValue, greenValue, blueValue); //sets right eye to a specific color  
rightEye(0, 50, 0); //make right eye green, 50 brightness value rightEye(0, 0, 0); //turn  
off right eye
```

This is a quick function to set right eye to a specific color.

.

leftEye();

```
leftEye(redValue, greenValue, blueValue); //sets left eye to a specific color leftEye(0,  
50, 0); //make left eye green, 50 brightness value leftEye(0, 0, 0); //turn off left eye
```

This is a quick function to set left eye to a specific color.

.

rightWing();

```
rightWing(redValue, greenValue, blueValue); //sets all pixels on wing to specific color  
rightWing(0, 50, 0); //make all pixels on right wing green, 50 brightness value  
rightWing(0, 0, 0); //turn off all pixels on right wing
```

This is a quick function to set all the pixels on the right wing to a specific color.

.

leftWing();

```
leftWing(redValue, greenValue, blueValue); //sets all pixels on wing to specific color  
leftWing(0, 50, 0); //make all pixels on left wing green, 50 brightness value leftWing(0,  
0, 0); //turn off all pixels on left wing
```

This is a quick function to set all the pixels on the left wing to a specific color.

.

Spirit Arduino Quick Reference

Light Sensors:

PIC_ReadAllAmbientSensors();

`PIC_ReadAllAmbientSensors();` //retrieves reading for all 3 ambient light sensors. Accurate to within 20ms ago.

Populates the following global variables:

ambLeft, ambRight, ambRear

.

PIC_ReadAllSurfaceSensors();

`PIC_ReadAllSurfaceSensors();` //retrieves reading for all 6 surface sensors. Accurate to within 20ms ago.

Populates the following global variables:

surfLeft0, surfLeft1, surfRight0, surfRight1, surfRear0, surfRear1

.

PIC_ReadAllAmbientSensorsInstant();

`PIC_ReadAllAmbientSensorsInstant();` //retrieves instant reading for all 3 ambient light sensors.

Populates the following global variables:

ambLeft, ambRight, ambRear

This function pauses PIC for a short moment to take a new instant reading of the sensors and reply with the values. Use this function if an instant reading is really necessary. In most cases, a reading that is between 1ms and 20ms old is okay. In that case, you should generally use the normal function `PIC_ReadAllAmbientSensors` instead.

.

PIC_ReadAllSurfaceSensorsInstant();

`PIC_ReadAllSurfaceSensorsInstant();` //retrieves instant reading for all 6 surface sensors.

Populates the following global variables:

surfLeft0, surfLeft1, surfRight0, surfRight1, surfRear0, surfRear1

This function pauses PIC for a short moment to take a new instant reading of the sensors and reply with the values. Use this function if an instant reading is really necessary. In most cases, a reading that is between 1ms and 20ms old is okay. In that case, you should generally use the normal function `PIC_ReadAllSurfaceSensors` instead.

.

PIC_ReadAllAmbientAverages();

```
PIC_ReadAllAmbientAverages(); //retrieves running average for all 3 amb light sensors.
```

Populates the following global variables:

ambLeftAverage, ambRightAverage, ambRearAverage

.

PIC_ReadAllSurfaceAverages();

```
PIC_ReadAllSurfaceAverages(); //retrieves running average for all 6 surface sensors.
```

Populates the following global variables:

**surfLeft0Average, surfLeft1Average, surfRight0Average, surfRight1Average,
surfRear0Average, surfRear1Average**

.

PIC_SetAverageIntervals();

```
PIC_AvgIntervalSurface = 6; //Average surface every 6th 20ms period.  
PIC_AvgIntervalAmbient = 6; //Average ambient every 6th 20ms period.  
PIC_SetAverageIntervals(); //Sets average intervals to PIC processor.
```

The PIC processor automatically calculates a running average based a given count of 8 previous readings. Set the global variables PIC_AvgIntervalSurface and PIC_AvgIntervalAmbient first, then call PIC_SetAverageIntervals(); to latch the values to PIC processor. A reading will be taken every n'th 20ms period, and averaged in with the previous readings taken at n'th 20ms periods. Setting the interval to a lower level will make the average change more quickly. Values can range from 1 to 255 periods.

Note: This function also latches the global variable PIC_AvgIntervalPower at the same time, which controls a similar average for battery voltage and current consumption averages.

Spirit Arduino Quick Reference

Servos:

.

servoCenters();

```
servoCenters(); //sets all 3 servos to default positions
```

This function sets all 3 servos to the default positions. These default positions are defined in the hardware.h tab as the values SERVO_TILT_CENTER, SERVO_PAN_CENTER, and SERVO_GRIP_STOWE.

The default values used by servoCenters() are as follows:

SERVO_TILT_CENTER = 100

SERVO_PAN_CENTER = 90

SERVO_GRIP_STOWE = 10

.

servoTilt();

```
servoTilt(position); //sets the tilt servo servoTilt(100); //sets the tilt servo to 100 degrees - facing straight forward servoTilt(30); //sets the tilt servo to 30 degrees - facing downward servoTilt(130); //sets the tilt servo to 130 degrees - facing upward
```

This function sets position of the Tilt servo. The position is generally given in degrees, with larger numbers tilting upward and smaller numbers tilting downward. The accepted range is from 0 to 180, though the software on the PIC processor does limit this to between 10 and 175 to prevent over-driving the servos.

.

servoPan();

```
servoPan(position); //sets the pan servo servoPan(90); //sets the pan servo to 90 degrees - facing straight forward servoPan(30); //sets the pan servo to 30 degrees - facing left servoPan(150); //sets the pan servo to 150 degrees - facing right
```

This function sets position of the Pan servo. The position is generally given in degrees, with smaller numbers facing toward the left and larger numbers facing toward the right. The accepted range is from 0 to 180, though the software on the PIC processor does limit this to between 10 and 170 to prevent over-driving the servos.

.

servoGrip();

WARNING: Be careful to avoid over-driving the grip servo. If an object is grasped and you attempt to close the grip servo more tightly, the servo will bind and will draw excessive current. Some minor binding is probably okay, but avoid attempting to close the servo all the way around an object - the servo is intended to lightly grip an object so it may be dragged around the surface.

```
servoGrip(position); //sets the grip servo
servoGrip(10); //sets the grip servo to 10 degrees - grippers fully retracted / stowed
servoGrip(125); //sets the grip servo to 125 degrees - grippers fully closed
```

This function sets position of the Grip servo. The position is generally given in degrees, with smaller numbers retracting the grippers, and larger numbers more fully closing the grippers. The accepted range is from 0 to 180, though the software on the PIC processor does limit this to between 10 and 125 to prevent over-driving the servos.

servoSpeed();

```
servoSpeed(speed); //sets servo speed approximately in degrees per second
servoSpeed(3000); //maximum servo speed
servoSpeed(20); //slow movement of servos
```

This function sets the speed at which the servos move, approximately in degrees per second. By setting a high value like 3000 servos will move as fast as they are physically capable. Setting lower values of 20 to 90 create smooth movements. Note that servo speed can be adjusted during a movement. All servo speeds are set to the same value.

PIC_ServosInMotion();

```
PIC_ServosInMotion(); //query to see which servos are in motion
servosInMotion; //will be not zero if any servos are in motion
servoMotion_Tilt; //will be "1" if Tilt servo is in motion, "0" if Tilt servo is not in motion
servoMotion_Pan; //will be "1" if Pan servo is in motion, "0" if Pan servo is not in motion
servoMotion_Grip; //will be "1" if Grip servo is in motion, "0" if Grip servo is not in motion
```

Because servos can be set to move in slow speeds, it may be useful to determine when a given servo has reached its target setpoint. This function can be used for this purpose. After starting a servo movement you can call this function to determine when some or all servos have reached their stopping positions.

Important Notes:

- This function will indicate the servo has stopped when the PIC processor is sending the signal corresponding to the intended end point, but the servo itself may not have actually physically reached this point yet, especially if servo speed is set high. This function is mainly intended to be used to determine when servos set at slower speeds have reached their stopping positions.
- Calling PIC functions in very fast loops may cause problems. It is suggested to delay 2

to 5 milliseconds using code like `delay(5);` between repeated calls to this function.

`gripGrasp();`

```
gripGrasp(threshold); //Causes grip servo to grasp an object
gripGrasp(30); //Very sensitive grasp
gripGrasp(250); //Very firm grasp
```

This is an experimental function intended to cause the gripper to close around an object, and stop moving when the gripper has grasped the object. It generally works okay. A few warnings and explanations will help you use the function. The gripper arms and servo don't have sensors to tell us when they reach an object. However, when the servo does reach an object, it begins to draw much more current because of the resistance. We can measure the current being drawn by the entire robot and look for spikes to this current, and use those spikes to guess whether a servo may have reached an object. But this is easier said than done, as each small movement of the servo does draw a large spike of current. When the servo reaches an object, these spikes will be larger and more consistent. This function attempts to measure this automatically.

- The threshold value given is the average current increase in milliamps that the function is looking for. Making the threshold very low will cause the grippers to stop moving before touching an object, and making the threshold too high will make the grippers hold too tightly to the object and cause binding of the servo.
- This function accepts threshold values between 0 and 250.
- If the grip servo continues to buzz after grabbing an object, you may wish to back off the position some after the grip completes. You can determine the position the servo stopped by reading the global variable `servoPos_Grip` which will have the last commanded position stored. You can then use code like `servoGrip(servoPos_Grip - 2);` to cause the grip servo to back off 2 degrees from its last stopping point.
- This function automatically moves the grip 2 degrees at a time and evaluates the current draw between movements.
- It is best to run this function with motors stopped and other servos stopped, as the current drain spikes from these other sources may confuse the readings of this function. It is also advised to turn off most or all of the NeoPixels while this function runs. Even NeoPixels turned on a low settings will cause current pulses that may interfere.
- If you're more advanced - you can find this function in the Hardware tab. Feel free to play with it or use as a basis for your own custom implementation.

Servo Trim:

Servos may differ somewhat and even when properly assembled, you may notice `servoCenters()` doesn't exactly center some of the servos. For this reason, the PIC processor has a trim setting for each servo, where you can lock in a small offset so the servos center to their correct positions and other relative movements from these positions are more accurate.

The Arduino processor has a small permanent memory called EEPROM where it stores the last trim values used. It is only necessary to set these trim values one time in any of your sketch programs and the trim value will be stored. From that point forward, you can use any sketch program and the previously set trim values will still be used.

To adjust the servo trim, edit the trim values. You can find these at the top of the Hardware tab.

```
volatile signed char servoTrim_Tilt =25 ; //Tilt servo trim (-127 to +127) volatile signed char servoTrim_Pan =0 ; //Pan servo trim (-127 to +127) volatile signed char servoTrim_Grip =-1 ; //Grip servo trim (-127 to +127)
```

These are the values that set trim for each servo. Setting the value to anything other than zero will cause the value to be stored in memory. This value will remain even after loss of power or re-programming with a new sketch. Do some trial and error with these values until you get your servos trimmed. If you want to zero out the trim, you need to use a value other than actual zero as that value is ignored - instead set a given trim to -1, which is very close to zero and the program will still update it as expected.

To manually set the servo trims, you can call the following functions:

```
servoTilt_Trim(trimValue); // send trim value for servo servoPan_Trim(trimValue); // send trim value for servo servoGrip_Trim(trimValue); // send trim value for servo
```

- These functions accept values between -127 and 127.
- Calling these functions will not update the EEPROM memory so you can experiment without altering the stored value.
- If you're more advanced - you can study the function `setServoTrim()` in the Hardware tab. This function handles the EEPROM storage and query functionality as well.

Spirit Arduino Quick Reference

Chirp:

.

playChirp();

```
playChirp(frequency); //plays piezo chirp at given frequency in Hz playChirp(2000);  
//plays piezo chirp at 2000 Hz playChirp(0); //turns off piezo chirp
```

This function tells the PIC processor to create a square wave to the piezo chirp at the given frequency in Hz. Calling this function in a fast loop, or calling it in a fast loop with different tone values may result in a clicking or pulsing sound from the piezo. This is normal and does not harm anything. This function may be improved in future versions of the PIC software to eliminate these clicks. (If anyone wants to volunteer to do this, please let us know - it will require some low level setting of the clock and multiplier for the PWM peripheral on the PIC processor).

.

offChirp();

```
offChirp(); //turns off piezo chirp. Same as calling playChirp(0);
```

Easy way to turn off the piezo chirp.

.

Chirp Notes:

- A tone may become "stuck" on during startup or shutdown of the rover. This is normal and does not cause problems. This happens because the PIC will continue playing a given tone until told to stop.
- The chirp is automatically turned off during the initialization sequence when the Arduino code begins running.

Spirit Arduino Quick Reference

Rangefinder:

.

PIC_RangefinderEnable();

```
PIC_RangefinderEnable(); //enables the rangefinder to measure range every 20ms.
```

This is the most common way to enable the rangefinder. Call this function at least one time before starting to read the rangefinder. This function can be called one time in your setup() function at the top of your sketch. This function is the same as calling `PIC_RangefinderAutoInterval(1);`

.

PIC_RangefinderDisable();

```
PIC_RangefinderDisable(); //disables the rangefinder, stops auto-measurements
```

This is the most common way to disable the rangefinder. This function is the same as calling `PIC_RangefinderAutoInterval(0);`

.

PIC_RangefinderAutoInterval();

```
PIC_RangefinderAutoInterval(n); //Starts rangefinder making range measurement every n'th 20ms PIC cycle.
```

Example:

```
PIC_RangefinderAutoInterval(3); //Starts rangefinder making range measurement every 3rd 20ms PIC cycle.
```

Starts rangefinder to automatically make a range measurement every n'th 20ms PIC cycles. Setting the interval to 0 will disable the rangefinder.

.

PIC_ReadRangefinder();

```
PIC_ReadRangefinder(); //returns range in mm from last auto range reading.
```

Populates the following global variables:

rangeFinder, rangeGoodCounts

This function is the most common way to read the rangefinder. You must call `PIC_RangefinderEnable()` at least once before this function will return good data. The global variable **rangeFinder** holds the range value in mm. Returned results will be between 0 and 1000.

The rangefinder needs to be nearly perpendicular to the surface being measured to get a good result (the rangefinder works by sending sound pings, which will bounce off in a different direction from surfaces at an angle). Each time the PIC measures a reading it

considers to be good and reliable, the value of **rangeGoodCounts** will increase by 1. It will increase to a maximum of 250 (good range measurements can still be made, but **rangeGoodCounts** won't increase any further). Each time the PIC measures a reading it considers to be bad, out of range, or not reliable, **rangeGoodCounts** will go back to zero. In most cases, the range measurement **rangeFinder** will also go back to zero.

When reading the rangefinder, evaluate the resulting value. If the value is zero it can be assumed that a good reliable measurement hasn't been made. This can happen because the rangefinder is at too great an angle of the object or surface being measured, or if the range is extremely small (holding an object within 20mm of the rangefinder), or the object is over 1000mm away from the rangefinder. To determine the value is good and reliable, you may also evaluate if **rangeGoodCounts** is above a threshold you may choose.

.

Spirit Arduino Quick Reference

Power Registers:

Several functions are available to monitor various status of the rover.

.

PIC_ReadPower();

```
PIC_ReadPower(); //gets voltage and power values from PIC processor powerVoltage //global variable, now holds battery voltage (in millivolts) powerCurrent //global variable, now holds total current consumption (in milliamps)
```

This function reads the key power values - battery voltage, and current consumption. This reading is accurate to within 20 milliseconds (the value returned is the last reading taken during the last 20ms cycle). The function automatically populates two global variables `powerVoltage` and `powerCurrent`. These variables can be used anywhere in your code. Each time this function is called, the variables are updated.

.

PIC_ReadPower_Instant();

```
PIC_ReadPower_Instant(); //gets instant voltage and power values from PIC processor powerVoltage //global variable, now holds battery voltage (in millivolts) powerCurrent //global variable, now holds total current consumption (in milliamps)
```

This function works the same as the normal `PIC_ReadPower`, except the resulting values are instant, and do not include the 20 millisecond delay. When this function is run, the PIC processor will pause for a brief time to perform an instant reading of the power values and replies back with the values. It is suggested to generally use the normal `PIC_ReadPower` function, though you can use this function if a more instant reading is required.

.

PIC_ReadPowerAverages();

```
PIC_ReadPowerAverages(); //gets average voltage and power values from PIC processor powerVoltageAverage //global variable, now holds average battery voltage (in millivolts) powerCurrentAverage //global variable, now holds average total current consumption (in milliamps)
```

This function returns the average readings of battery voltage and current consumption. This is a running average from the previous 8 times the PIC processor performed its 20 millisecond interval measurement. You will find this value is more stable over time than the other readings above as the average tends to filter out spikes.

Spirit Arduino Quick Reference

UART:

The PIC processor on the Spirit Mainboard is able to relay UART messages from the Arduino processor to several possible end points. The UART should be initiated by setting the desired communication path and setting the baud rate.

PIC_SetComUARTMode();

```
PIC_SetComUARTMode(); //sets the target device for UART communication  
PIC_SetComUARTMode(0); //WiFi Mode - Sends UART traffic to the ESP8266 module socket  
PIC_SetComUARTMode(1); //BlueTooth Mode - Sends UART traffic to the HC-06 module socket  
PIC_SetComUARTMode(2); //XBee Mode - Sends UART traffic to the XBee module socket  
PIC_SetComUARTMode(3); //Pi Mode - Sends UART traffic to UART port on the Raspberry Pi
```

This function configures a selector switch on the Rover main board to communicate with the desired UART target. Transmit and Receive traffic will go to/from the intended UART target until the mode is changed.

Spirit Arduino Quick Reference

Advanced Users:

This section is for advanced users who want to experiment further on their own.

Advanced Communication:

Have a look at the Comms tab in the base sketch for all the available communications functions. This is where you will find the core functions described in this guide. You will also find some functions not described. We will document these further eventually, but you can likely figure them out on your own with some trial and error.

Spirit Raspberry Pi Setup

This section describes the setup of the Spirit Raspberry Pi unit.

Spirit Raspberry Pi Setup

Initial Configuration:

.

This guide assumes a fresh install of Raspbian OS on your Raspberry Pi. The card shipped with the Spirit Rover already has this OS installed. It can also be downloaded and added to your own microSD card following instructions at the official Raspberry Pi website.

Advanced users can follow these instructions or use their own. This is a general guideline to get your Raspberry Pi setup and ready for use on the Spirit Rover.

For these steps you will be using the Linux based operating system on your Raspberry Pi. You will do most of these steps using a terminal or console window without a graphical user interface. This is how the power users do it. It can be a bit complicated and overwhelming if you've never worked with a terminal window before. Just hang in there and try to type the commands exactly as shown. You may need to use google and watch some YouTube videos if you need help with the specifics. We'll try to make the guide as user-friendly as possible, but as you will soon discover with Linux and more advanced code, you will need to be resourceful in finding your own answers to some questions. This is how all the power users have learned and they all feel a bit overwhelmed at first.

VERY IMPORTANT NOTE: The Raspberry Pi draws a lot of current from the battery. If you begin working on the Pi and don't have your Spirit charged, you may find the Spirit wants to turn off due to low battery after a while. You may want to leave the Spirit plugged into a USB connection for a while before starting. The orange CHG (charge) LED next to the battery connector will remain lit during charging and will turn off automatically when charging is complete. You can operate the robot during charging, so you may want to leave this plugged in during the setup process as it may take a while.

.

Startup and WiFi Setup:

The first step is to startup the Pi and connect to WiFi. This will allow you to remotely access the Pi from another computer, which is where you will do most of your interaction from.

For the first startup, we suggest attaching an HDMI cable to the Pi (this can all be done while it is installed in the Spirit robot). Also attach a mouse (optional but useful for the very first step) and a keyboard. We suggest using corded mouse and keyboard. The wireless dongles

used for wireless keyboards and mouse may pull more current from the device than necessary and may cause it to shut down).

Raspi-Config

On first boot, the Raspberry Pi will likely boot into the graphical desktop environment.

Open a terminal window (it is the black monitor symbol in the upper left corner).

Start the Raspberry Pi configuration utility with this command:

```
sudo raspi-config
```

Make the following changes:

(Optional)

Select Internationalisation Options and set I4 Change Wi-fi Country options if needed.

(Required)

Boot Options > B2 Text Console Autologin Text Console as 'pi' user.

Advanced Options > A4 SSH > Enabled, Module Loaded By Default > Yes

Advanced Options > A7 I2C > Enabled, Module Loaded By Default > Yes

Once these have been set, select <Finish> and reboot when prompted. If you miss it or are not prompted, you can goto the menu and select shutdown and reboot.

After reboot, you should see a screen with lots of text scrolling as the Pi reboots. When complete, you will be presented with the standard Linux prompt. pi@raspberrypi: \$

Keyboard

The Keyboard setup is massively over-complicated. If you discover your keyboard won't type double-quotes, you will need to manually change the keyboard setup. There are options to do this in the raspi-config above, but you're presented with a few thousand options and none of them seem to work (at least not with any of the keyboards I own). If this is your situation, perform the following:

At the command line, type the following:

```
sudo nano /etc/default/keyboard
```

Edit the file that appears, being careful to not delete the existing double quotes. Make your file look like this:

```
XKBMODEL="pc101"
```

```
XKBLAYOUT="us"
```

```
XKBVARIANT=""
```

```
XKBOPTIONS=""
```

```
BACKSPACE="guess"
```

To save the file press Control-X to exit, then press Y to save changes, then press return or enter. This should return you to the command prompt with the file saved.

You can verify the file is correct by typing:

```
cat /etc/default/keyboard
```

This should display the file in the terminal window.

Reboot the Pi by typing `sudo reboot`

After the reboot, your keyboard should now work. You can test this at the command prompt after the reboot by trying to type a double-quote at the command line. If you get a double quote, then you're good. If you're still getting an @ symbol or something else, you'll need to try a different configuration. Try google for this (as I said, this step is massively over complicated, the first time I used a Pi it took me hours to get just this part working).

Set Pi to Connect to WiFi

In this step we will set your Pi to automatically connect to your WiFi network so you can access it from a different computer, like your normal PC or Mac, etc.

Edit the file that controls access to WiFi networks. Enter the following command at the command prompt. Enter it very carefully. If a blank document comes up with no content, then you have likely made an error typing the name and path.

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Under the text already present in the file, add the following. Note the curly braces. If this is not correct, it won't connect to your WiFi. Enter your network SSID (the name of your network) and your WiFi password where indicated.

```
network={
  ssid="your wifi ssid network name"
  psk="your wifi password"
}
```

Press Control-X, then Y, then Enter to save the file. You can verify the file by typing:

```
sudo cat /etc/wpa_supplicant/wpa_supplicant.conf
```

Reboot the Raspberry Pi with `sudo reboot`

After the reboot, verify your Pi is connected to your WiFi network by typing:

```
ifconfig
```

Look at the section next to "wlan0". If it is connected, the second line of this section should show the IP address your WiFi router assigned to your Pi. It will say `inet addr:192.168.1.10` (or something similar). The address should end in something other than .1 Make note of this IP address. Note that it may change from time to time, as this address is assigned by your

WiFi router, and it may occasionally change up addresses depending on what other devices you have connecting to your WiFi. The IP address is just the number part with the periods between. In the example above, the Pi has the IP address of `192.168.1.10`

These instructions should work in most cases. If you still have trouble, you'll need to spend some time on the internet searching for how to connect your Raspberry Pi to WiFi. You can connect your Pi through the graphical interface (which you can re-launch at any time by typing `startx` at the command line), but the Pi may not always automatically re-connect on boot.

To verify that your Pi can be found and connected to from your WiFi network, use the computer you plan to use for working on your Spirit and download the [Adafruit Pi Finder](#) application. Select the correct version for your operating system from the link.

When you run the application you should get a small window with the Adafruit flower logo. Click the button to find your Pi. It will search for a minute and hopefully come up with a result. The IP address displayed should match the address you saw in the `ifconfig` step above. You can then exit the Pi Finder application.

At this point, you should be able to connect to your Spirit Pi unit from another computer.

Connect using SSH and SFTP

Note: When logging into your Pi, the default login details are Username: `pi` Password: `raspberrypi`

You will generally connect to your Pi using two pieces of software. The first piece of software is called an SSH Client, which is software you will install on your computer (the one you will use to work with your Spirit). If you google "SSH Client" you will find lots of options. I suggest **MobaXTerm** for PC users. It is fairly full featured and free.

SSH basically presents a console screen on your PC (or Mac, Linux, etc) computer that looks just like the terminal window you have been using. It works the same as typing directly into your Pi with an attached keyboard. This allows you to type all sorts of Linux commands to the Pi, and this is how power-users interface with all kinds of servers - everything from Raspberry Pi to web servers and the like. Learn as much as you can about this console / terminal operation as you can. It will be useful as you do other things with Linux in the future.

While SSH allows you to type commands, it's not user friendly for moving files to and from your Pi. For this function we will use a different piece of software called an SFTP client. SFTP stands for Secure File Transfer Protocol. Like SSH clients, there are many options available. I suggest using **WinSCP** for Windows users. It is free and fairly user friendly. This program creates a window on your PC that has your local computer file system on one side,

and the file system on your Pi on the other side. You can drag and drop files in either direction between your PC and the Pi. When you start writing your own python code, I find this is the easiest method to move your code files onto your Pi.

Download and install an SSH client and SFTP client onto your working computer, and attempt to connect with both to your Pi. You may need to dig around the net a bit for specifics on this. Google and YouTube are good resources for this.

If you can connect via SSH and see a command prompt in the terminal window (which will look just like the window you worked in above when typing other commands directly into the Pi) then you are all set. Same thing with connecting via SFTP.

Spirit Raspberry Pi Setup

Pi Software Install:

The Pi should be running the Raspbian Operating system. This operating system includes the base software needed to run the Pi. However, we will be doing some other things with our Pi that require some extra bits of software to be added.

To install this software, complete the steps in the previous section "Initial Configuration", then connect to your Pi using an SSH client.

Perform the following steps in order, and carefully pay attention to exactly how the commands are entered.

Update Packages:

First we will make sure all the software on the Pi is updated to most current versions. Type the following commands, one at a time and let each complete before doing the next. They may each take a while to complete. Sometimes they appear to hang, but just give them time and they should complete and return you to the normal command prompt.

NOTE: If the Pi does hang for a long time and you loose connection via your SSH client, make sure the Spirit power is still turned on (the left most blue LED named "PWR" should still be turned on). If you have problems with power turning off, plug in the robot via USB for a while and let the battery re-charge then begin from the previous successful step in your setup).

Update Packages:

```
sudo apt-get update
```

```
sudo apt-get dist-upgrade
```

 (when prompted for space, type Y to confirm)

```
sudo apt-get clean
```

Install Git Core:

```
sudo apt-get install git-core
```

Install i2c-tools:

```
sudo apt-get install i2c-tools
```

Install python-smbus python module:

```
sudo apt-get install python-smbus
```

Install smbus2:

```
sudo pip install smbus2
```

Set I2C bus baud rate to 400kHz:

Edit /boot/config.txt with `sudo nano /boot/config.txt`

Use arrow keys to move down to the very bottom of the file and enter the following new line to the file:

```
dtoverlay=i2c-arms-baudrate=400000
```

Press Control-X, then Y, then Enter to save the new version of the file.

Set up Python and set up a new VirtualEnv (virtual environment) to run the Spirit Rover python code:

Install the 2.7-dev python module:

```
sudo apt-get install libpython2.7-dev
```

 (Press Y to continue after the disk space check)**Install NeoPixel Code:**

```
sudo pip install rpi_ws281x
```

Install virtualenv for python:

```
sudo pip install virtualenv
```

Set up a new virtual environment for the rover software:

```
virtualenv --system-site-packages rover
```

It is very important to include the `--system-site-packages` in the command. This will create a new directory and python executable in `~/rover/bin/python`, and allow the new virtual environment to use the main system packages already installed.

Configure the Pi to start running python inside this new virtual environment automatically when the Pi boots:

Edit the `.bashrc` file. This is a script file that runs at each login.

```
nano .bashrc
```

 (Note the period before the file name)

Go to the end of this file and add the following line:

```
source $HOME/rover/bin/activate
```

Exit and save the file with Control-X, T, then Enter

Reboot the Pi:


```
sudo reboot
```

This will drop connection in your SSH client. Wait a minute or so and re-connect a new SSH session. You may need to let your Pi boot for a minute or two before you can establish a new SSH session.

When you log back in, you should see the following prompt:

```
(rover) pi@raspberrypi:~ $
```

Note the command line now starts with `(rover)` This indicates you are working in the new virtual environment created for your Spirit Rover code.

Setup the Plum Geek Pixel Server and Shutdown Monitor on the Pi:

The pixel server is a control system we wrote ourselves for the Spirit rover. Controlling the physical pins on the Raspberry Pi processor requires "root" access, and communicating with the NeoPixel LED lights requires signals from these physical pins. Normally, you will log into the Pi as user "pi" which does not have root access. To get around this, we kick off a script that runs in the background, which in turn controls the pixels. You can use your normal user code (run by the user "pi" or any other user) to talk to this pixel server and control the lights. The pixel server also allows for doing cool automated fades and simple color animations automatically in the background. This section describes how to install and setup the pixel server.

The Shutdown Monitor watches a special pin that is triggered by the Spirit Mainboard just before the unit powers off. When this line is held low for about 1/2 second, this shutdown monitor script causes the Raspberry Pi to immediately shut down. It is important to allow Linux computers to gracefully shut down (rather than just pulling the power) to avoid possible problems with the file system. Once this shutdown monitor script is installed it works in the background and you can power the Spirit on and off without worrying about it.

PG Pixel Server:

Copy the folder `pgpixelserver` into the `rover` folder you created earlier. Use an SFTP client for this. The `pgpixelserver` folder should have five files inside it.

After the folder and contents are copied, you will need to make the launcher script file executable.

Make both launcher script files executable:

```
chmod u+x ~/rover/pgpixelserver/pgpixelserver_launcher.sh
```

```
chmod u+x ~/rover/pgpixelsever/shutdownmonitor_launcher.sh
```

Create a directory for pixel server logs:

```
mkdir ~/rover/logs
```

Add entry to Cron to start the Pixel Server and Shutdown Monitor on boot:

Start the crontab editor. (Make sure you use "sudo" here, so you will be editing the root crontab, which is important for allowing the neopixel code to access the physical control pin on the Pi):

```
sudo crontab -e (Select option 2 "nano" from prompt)
```

Move to the bottom of the file and enter the following lines. The first is just a note, the second is the line that is actually executed. You may want to copy and paste this into the editor, it is fairly long and complicated and needs to be entered exactly as shown.

(Note, the gitbook may wrap the following lines. The code below should be entered as 3 separate lines. The first starts with the # symbol, and the two starting with @reboot are complete lines - the end parts 2>&1 may wrap in the gitbook, but they should be entered on the same line as the beginning @reboot).

```
# start PG Pixel Server and Shutdown Monitor on boot
```

```
@reboot sh /home/pi/rover/pgpixelsever/pgpixelsever_launcher.sh  
>/home/pi/rover/logs/cronlog 2>&1
```

```
@reboot sh /home/pi/rover/pgpixelsever/shutdownmonitor_launcher.sh  
>/home/pi/rover/logs/cronlog 2>&1
```

In Linux, crontab is a system that automates background tasks. By issuing the @reboot instruction we are telling cron to execute this line each time the Pi is booted. The line causes Linux to run the pgpixelsever_launcher.sh script, and place any logs or errors into the logs directory we created earlier. This will launch the pixel server in the background.

After entry is made, press Control-X to exit and save.

Reboot the Raspberry Pi, then test to make sure the pixel server started up without problems.

```
sudo reboot
```

After reboot, log back into the Pi and lets check a few things.

```
cat ~/rover/logs/cronlog
```

This command prints the contents of the cronlog. If everything went as expected, this log should presently be empty, so the above command should not produce any output. If there was a problem starting the pixel server, you will likely find some cryptic errors displayed from this file.

Try one more quick check:

```
ls -la ~/rover/pgpixelserver
```

This will list the contents of the pgpixelserver directory. If the pixel server started up, you should now see a new file in this directory named PixAnimation.pyc (note the "c" on the end). If this file is present, it indicates the pixel server has started up at least one time.

If both of the above checks are good, you should have a happily running pixel server from this point forward.

You can verify the shutdown monitor script is working correctly by logging into an SSH session, then press and hold the Spirit Power button (PWR) until the PWR LED begins to blink (this is the blue LED on the far left). Once the shutdown sequence has been initiated, you should see the following line broadcast to your SSH terminal just before connection is lost:

```
Spirit Shutdown Now!
```

If you have any problems with either of these scripts, go back and make sure your entry in the crontab is exactly correct, and that you have correctly made both launcher scripts executable in the steps above.

.

Add Actual Python Code to Spirit:

Now that we've done the bare bones setup, we can finally add a directory with some code!

Copy the folder `democode` into the `rover` directory you created earlier. Use SFTP to accomplish this.

Important Note: When using the Pi to control your Spirit Rover, it is suggested to load the Arduino sketch "Spirit_BaseSketch_PiControl" onto the Arduino processor. This sketch just waits for motor control signals from the Pi and doesn't do anything else. You can add your own code to this Arduino sketch as well if you like, but generally you will control the robot from either the Arduino processor or the Pi.

There are two important files in the democode directory: `spirit_core.py` and `spirit_pixels.py`. These files should be present in any directory where you run other spirit python files from.

The other files in this directory are code examples that can be run and edited.

You can run the examples like this:

```
python volts.py
```

This will run the "volts.py" script, which displays battery voltage in your terminal window. Try the others and see what happens.

Most of the important functions are contained in the `spirit_core.py` file. They generally work the same as similarly named functions in the Arduino base sketch. You can have a look over this core file to see what functions are available. **Generally you should use the functions that are in all lower case lettering. Some functions contain upper case letters, but these functions are generally used by other functions within the core file and generally shouldn't be called on their own.**

This is the end of this section for now. We will update this and add more detail about the use of the functions soon. This is just to get the more advanced users started.